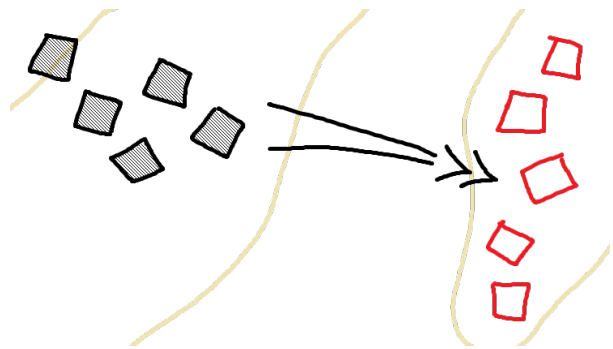
rat-scramble

a bit of an exploratory thing i suppose

the intent

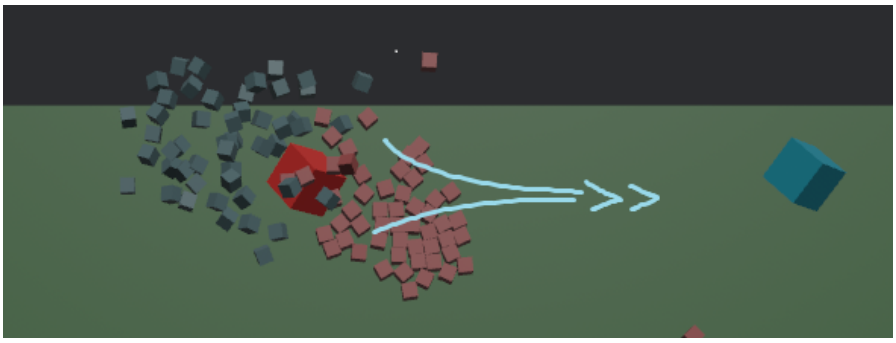
Having played a few too many wargames lately, I decided it would be a good idea to implement a basic skirmish-level real-time physics-based battle simulator. I'd be using the Bevy game engine, which as I understood it was less like some of the mainline game engines with viewports and gameplay frameworks, and more like a GUI-less implementation of the Entity-Component-System pattern written in Rust.



Woah, is that a daring military maneuver unfolding before our very eyes? Those rats must be super well trained or something! Yowzers!

Ideally, it would model simple maneuvers by having each team with either WASD or Arrows controlling a cursor their rats would follow. Something vaguely recognizable as a contribution to the wargaming genre, but mostly just a novelty physics toy for my itch.io page, which needs more than zero fun little novelty toys on it!

Yes, I do game dev for fun. No, I don't have any silly little projects you can hold that don't involve a huge team of other people. Like- What?! I simply **must** fix that!



No rats were harmed in the making of this feature film.

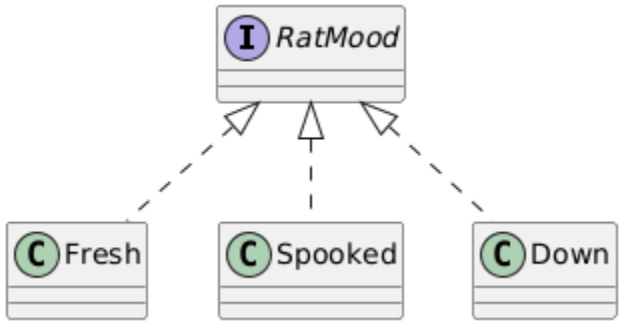
Now, I didn't realize this would turn into a growth narrative. But here's what I tried first. I begin by disrespecting Rust, suffering, and then rewiring my entire brain while I pull everything together.

If you get bored, skip to page 9 (**"moving rats"**). It's where I start having fun, at least.

plan a

So I formulated a very "object-oriented programming" sort of plan, not bothering to check whether Rust was an object oriented language (it's not.) The basic game plan was to implement three different states for each rat representing their battle readiness and fervor.
(See Bob Nystrom's article on the State pattern here.)

And the Rat would change state between these different RatMoods as follows:



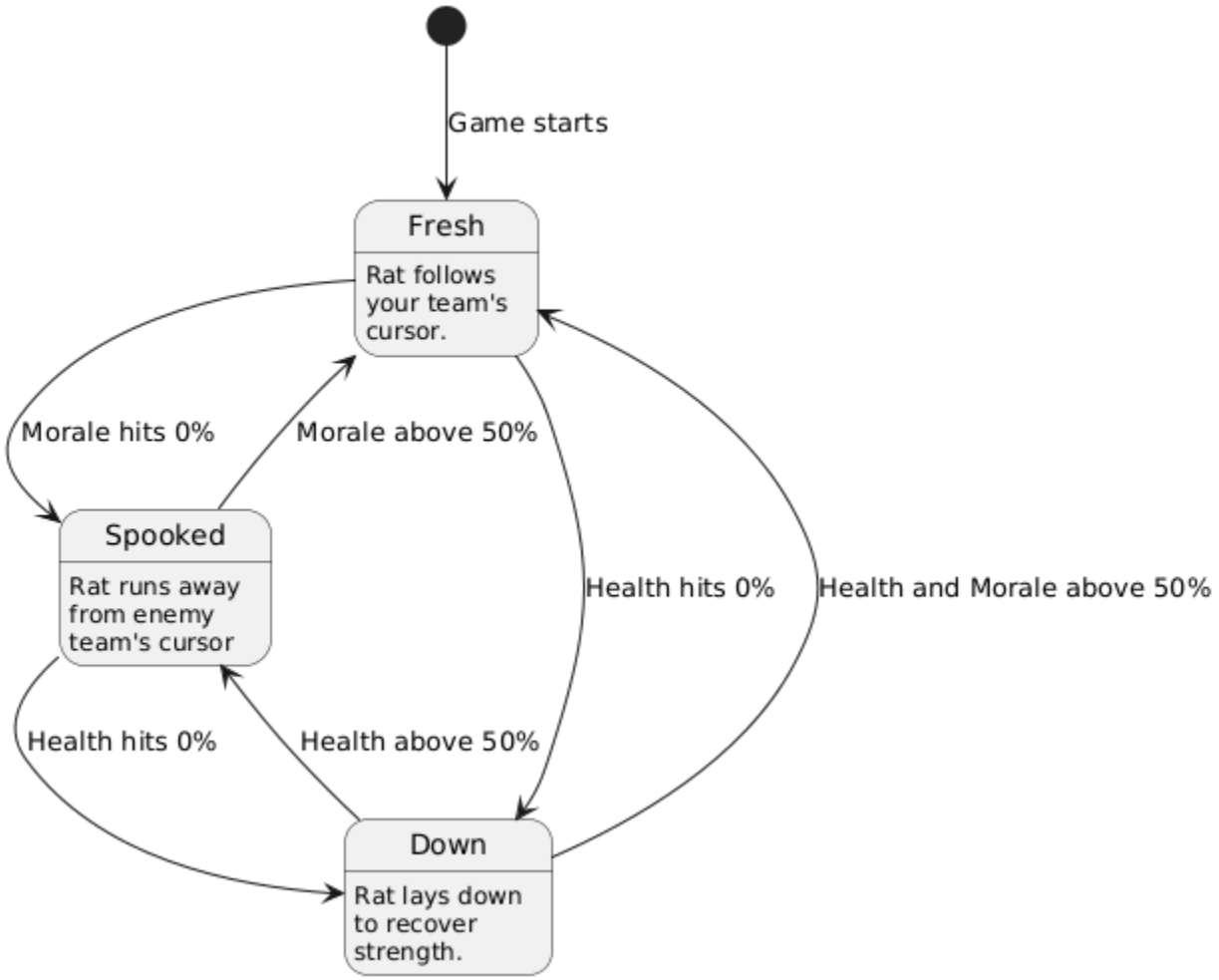
Fresh – The rat has enough Health to remain standing, and enough Morale to continue fighting.
Spooked – The rat has enough health to keep standing, but no morale, and will refuse to fight.
Down – The rat does not have enough health to remain standing.

And we'd give each rat health and morale resources that, when raised or lowered, would trigger these state changes.

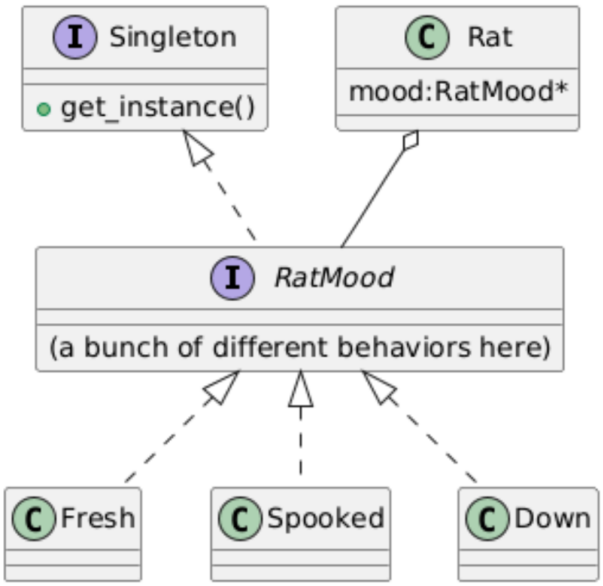
And because I didn't think spring physics in games was going to be jank enough to sell that these rats are really silly, I used a PID controller to move the rats around, which turned out to work great.

rat moods and semi deep weeds

Internally, I was calling the collection of states “rat moods”. Here’s how they worked:



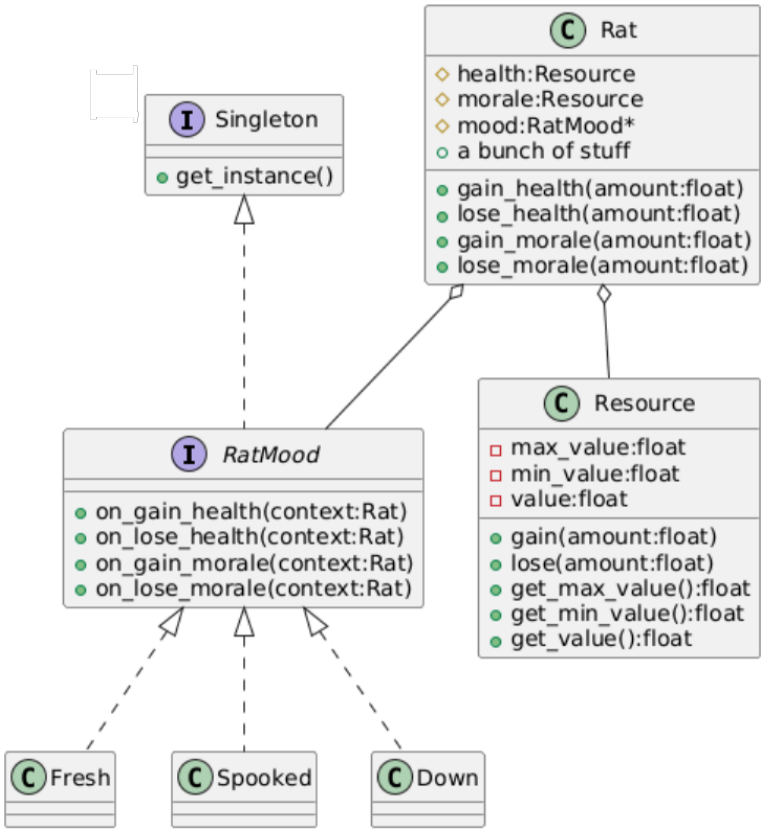
As a bit of an optimization, I’d make it such that we wouldn’t need to create and destroy a bunch of different state objects whenever we wanted to get anything done – we’d only ever create one copy of `Fresh`, `Spooked`, and `Down` respectively, and simply tell each `Rat` object which one corresponded to their mood at any given time.



That is- we'd give the `Rat` a pointer to a `RatMood`, and make each `RatMood` a Singleton. (Bob Nystrom also has a great article on Singletons – especially on how *not* to use them – here.)

And then we have to elaborate on this UML Class diagram with some of the funny little bells and whistles like Health and Morale, which I'd encapsulated in a funny little thing called "Resource" so we keep the values between 0% and 100%.

Which expands the UML diagram out to something like this:



As I understand it, Rust prioritizes performance, parallelism, and safety, and doesn't really care for silly little things like making polymorphism easy.

I happened to be using RustRover, and it gave some very nice linting and hints and just said "dude use an enum what are you doing"... Except that's what I was trying to avoid! With states, you can just add an additional state without ever touching the `Rat` class, and without updating every switch statement you've ever written!

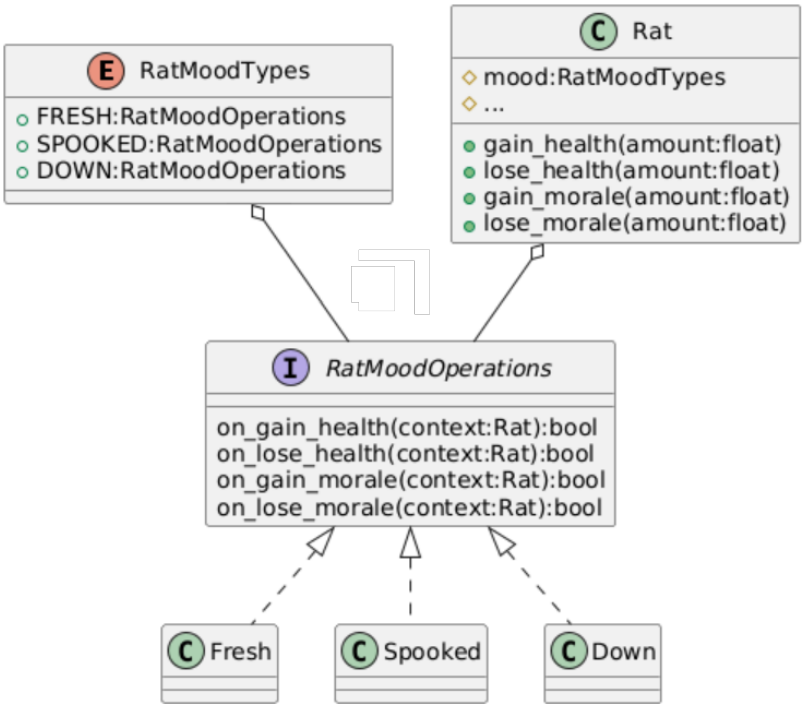
It's genuinely an incredible bit of object-oriented tech!

And now I have to update an enum every single match case that uses it whenever I want to add another state? How far we've fallen!

ratmood and what i tried

I hadn't totally figured out yet that polymorphism was an afterthought in Rust yet. let's go over what I tried. The faint of heart or judgemental programmer should skip this section, this is going to get ugly.

First things first – I threw out Singletons. Singletons are nice! People have done them in Rust before and they work great. It just wasn't mission critical so meh, whatever.



Then I broke up the `RatMood` trait (because Rust doesn't have interfaces, silly me!) into an enum so that `Rat` could hold onto it without the "`dyn`" keyword, and a trait that actually lets other classes implement shared functionality.

So... traits in Rust are for guaranteeing shared functionality between two structs, I'm just kind of silly and assumed that meant traits were as powerful as the interfaces I had worked with in other languages. So of course I didn't figure out how to do it for my use case. Something, something... "size of vtable cannot be known at compile time" if I remember correctly? I still don't *fully* understand the difference between a trait and an interface, but believe me when I say *there is a difference.*

Anyways. This change I made just moved the problem to a different spot.

Each of the `RatMoodTypes` still refer to a `RatMoodOperations` instance, which required the "`dyn`" keyword, which apparently was very cross with me. So- yeah.

It didn't really work, so I changed to just have `RatMoodTypes` implement `RatMoodOperations` instead.

I did stuff like this a few different times until I finally landed on this for my implementation of each of the three states:

```
// FRESH
2 usages  new *
fn fresh_on_gain_health(context: &mut Rat) -> bool { false }
1 usage  new *
fn fresh_on_lose_health(context: &mut Rat) -> bool {
    if context.health.get_value() <= TO_DOWN_HEALTH {
        on_exit(context);
        context.mood = RatMoodTypes::DOWN;
        on_enter(context);
        return true;
    }
    return false;
}
1 usage  new *
fn fresh_on_gain_morale(context: &mut Rat) -> bool { false }
1 usage  new *
fn fresh_on_lose_morale(context: &mut Rat) -> bool {
    if context.morale.get_value() <= TO_SPOOKED_MORALE {
        on_exit(context);
        context.mood = RatMoodTypes::SPOOKED;
        on_enter(context);
        return true;
    }
    return false;
}
```

That's right. The individual state classes do not exist anymore. They're prefixes to functions. They encapsulate nothing because they don't exist. Instead of `Fresh` owning a function called "`on_gain_morale(/*...*/)`" it's a standalone function called "`fresh_on_gain_morale(/*...*/)`".

We'll call them "aspirational classes", because *boy do I really wish they existed!*

Outside of the file rat_mood.rs, you just called "`RatMoodTypes::on_gain_morale(/*... */)`" and called it a day. It worked just like the State pattern! Here's how that looked:

```
// TODO reimplement this without a bunch of switch cases T-T
// TODO try to adhere to the state pattern
new *
impl RatMoodOperations for RatMoodTypes {
    new *
    fn on_gain_morale(context: &mut Rat) -> bool {
        return match context {
            FRESH : &mut Rat => fresh_on_gain_morale(context),
            SPOOKED : &mut Rat => spooked_on_gain_morale(context),
            DOWN : &mut Rat => down_on_gain_morale(context),
        }
    }
    /* other stuff here */
}
```

It was pretty slick and turned out to be loosely coupled in practice, but knowing that I wasn't supposed to need an enum on a thousand match statements to implement the state pattern just felt pretty bad. Man, I miss polymorphism!

Rust Fact!

You can pass functions as parameters. It's genuinely really darn powerful. So I tried to engineer a situation where I pass in both the calling object *and* a function that belongs to the calling object's interface `RatMoodOptions`, but uh. The problem there is twofold.

- There's no way to specify which trait or struct a function belongs to when it's being passed as a parameter. I checked the Rust book, there's no syntax for it!
- *This is just the same bad polymorphism again in a different spot!*

And I thought I was clever!

I have no doubt there's a better way to do this in Rust, but I'm not going to spend too much longer thinking about it at this juncture. Believe me when I said I tried a *lot of things*. I've already got more ideas, but when I learn more I'll revisit this.

the big lie

What I do know, is that I suffered at least twenty hours of pain on specifically this polymorphism thing, and I have a vague appreciation for Rust as something powerful, useful, and *not object-oriented*.

Like- nobody told me it *was*. I just kind of assumed.

Those diagrams? All lies. I'd been thinking of it all as though it was C++ or Java or whatever, over in Object Oriented land where these things just work. Or at the very least like Python, which is more procedural but includes so many nice features like properties, class methods, interfaces, code execution on unpickling (why not)... and you can just fake it!

But bringing this thought process into Rust has always been delusional!. There's no for interfaces in Rust, there's no for classes. I've been using them to describe traits and structs respectively. It's really simple to explain them as interfaces and classes! They aren't as powerful because of Rust safety features, though I couldn't tell you specifically what safety rules precluded my use case. I could have done this all very quickly in a system like Unity or Unreal Engine 5 and saved myself some trouble, since they work with those principles.

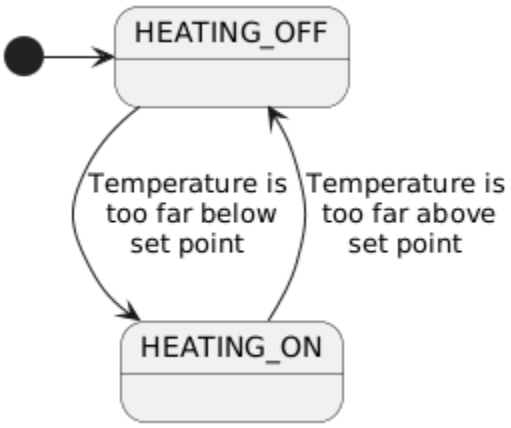
Being university trained in object oriented principles, and a tutor teaching them myself, I figured "oh I know how to do this! That's pretty straight forward, I'll have a delightful time learning Bevy. Rust is the most beloved language for 8 years in a row now on GitHub! It's great, isn't it?"

I think it's great! I will say, I had a dreadful stomach ache the entire time so that certainly colored my perception of the process. I'm writing in Rust for other projects now, so clearly no lasting harm occurred.

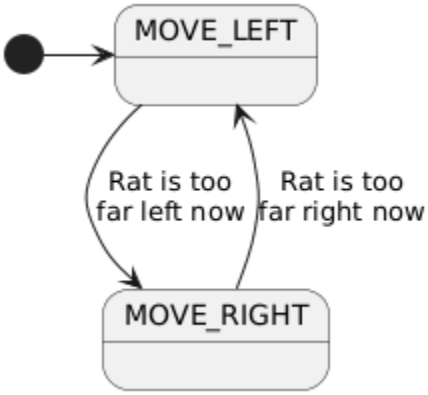
For a change of pace, let's talk about something that worked.

moving rats

I swear this is related. It's also my favorite part.



Consider the thermostat in my parent's house, which is only hooked up to the heaters. Its job is to keep the heat on until the temperature inside is warm enough, then turn the heat back off until it's needed again. It's a remarkably simple control system, and for its application it works really nicely.



Now- this is a pretty funny idea that we can actually apply to the rats, if we just consider one axis at a time.

The way I'm moving rats is by applying forces. If we're naively applying a force towards where the rat is supposed to be, they'll never settle until we add friction. And even then – there's no in-between state. This is fine for a thermostat – a building is usually pretty big, and will soak up heat slowly. If we over or under-shoot our target temperature, it likely won't be by much because the heater isn't that strong relative to the volume it needs to heat.

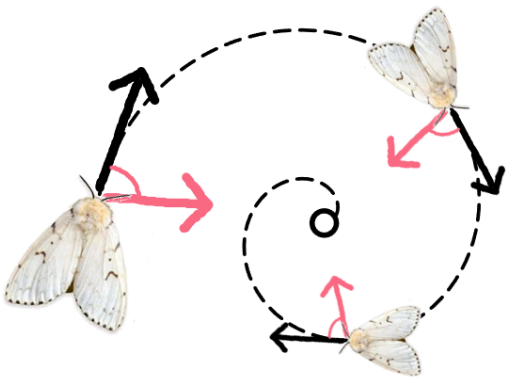
However, since rats are supposed to get places at more than a glacial crawl, we'll have to do something a bit more dynamic to really sell that these are organic creatures!

so many controllers to choose from

Cameras are usually mounted on a spring to give it a pleasant lag, and that was absolutely an option for the rats as well. I just didn't do it.

We can also just make the rats always keep moving, and have them face a point just to their left of the cursor. This would have produced a slow moving tornado reminiscent of schooling fish, churning around the cursor.

We may also simply have the rats hold the cursor at a certain angle from forward, which would be an implementation of Augmented Proportional Navigation guidance – which is really useful for moths who are wayfinding using the light of the moon. And explains why half of the moths that see a light that's nearby wind up converging on it. The other half spirals *away* from the light, but of course we never see them.



I decided to use the PID controller, which stands for Proportional-Integral-Derivative controller. It's used in cruise control systems, airplane autopilots, industrial process control, robotics, &c. I've also implemented them a number of times, so I felt comfortable doing this under time pressure.

how does a pid work?

There's plenty of mathematically rigorous explanations of the PID controller out there, so I won't dive too deep into the calculus here. But we'll need to define some terms.

		Parents' Thermostat	Rat
Process Variable	The variable we're trying to control from an external system.	Temperature	Our rat's Z-position
Set Point	The value we want our process variable at.	68°F (or 20°C)	Player cursor's Z-position
Error	The difference between our process variable and set point.	a degree or two	a few meters
Control Signal	The output of our controller, for connecting to our external system.	Powering the heater	Force applied to our rat

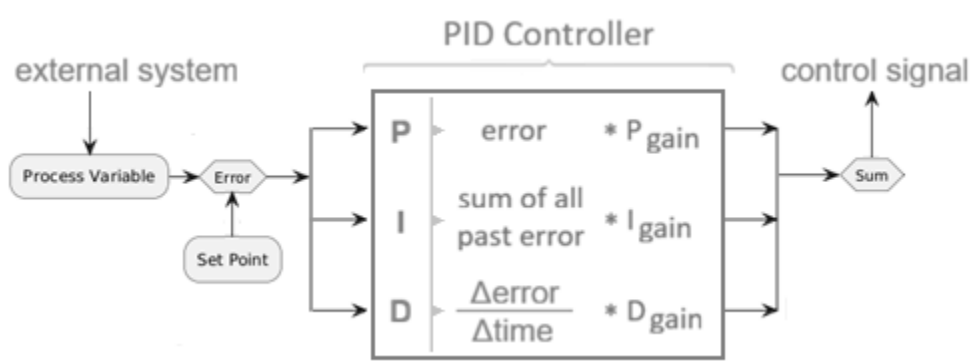
Error is the big one. The goal of a controller is to bring it as low as possible and keep it as low as possible.

Unlike the thermostat's controller, a PID controller can output any signal from 0 to 1, not just a simple ON and OFF, to reduce overshooting. It computes it by adding together three different components:

Proportional	Equal to the current error.
Integral	The sum of all previous errors, multiplied by how long it's been that way.
Derivative	Equal to how fast error is changing.

And each component is multiplied by a special "gain" factor. How do we come up with the gain? That's also an open area of study, but basically different methods work for different systems. I just pick them manually.

What I do usually is keep the values as low as possible, and slowly inch them up. What I actually did is set everything to 0.02, checked it once, and decided to roll with it even though the overcorrection was pretty aggressive.



I set the integral component to a negative gain, so it would actually slightly *magnify* any long-standing errors until the other components successfully corrected for it. This was to prevent them from staying clumped together during combat, which actually slightly worked.

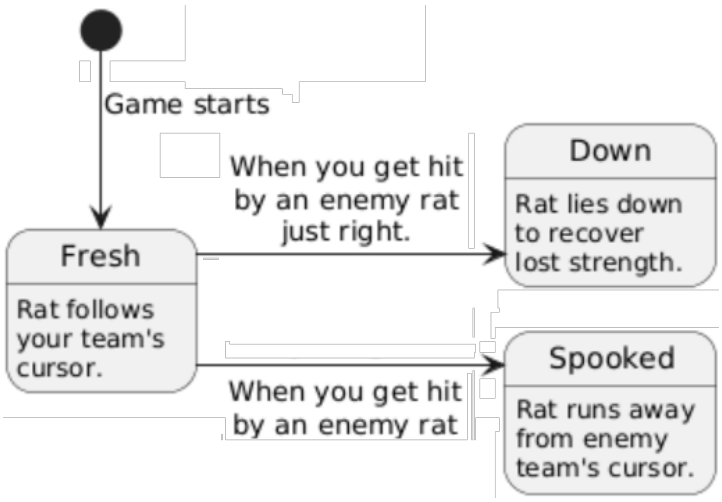
the catches

It works, just with its own catches, of course. In rat-scramble, the derivative component isn't getting smoothed out in the same way its counterpart in an analog electrical system might be, or a professionally designed PID controller in a vehicle. So whenever anything really weird happens with delta-t physics the derivative term is just absolutely screwed for a timestep.

Also, rats will on occasion run in wide circles around the main group before settling down. They get the zoomies, it's kind of cute.

the fate of ratmood

This does not work as previously advertised. The state transitions simply do not work the way I've described. Again, I'm a fraud. The Spooked and Down states work great. I just didn't add regeneration when rats are near their allies.



So this is what we got in practice.
I've slept since I last wrote this so it should be an easy fix, but this is extremely funny to me. Yes, state transitions work.

I should really add a better damage system than "deal damage to rats within range based on my PID's output magnitude". Because it's not amazing. And it's really laggy for some reason when they take damage?

Given that the very bare minimum I set out to achieve was to implement the state pattern, and I kinda implemented it and it technically worked kinda... it's not really so bad.

so do you like bevy?

Yes, actually I do. Don't try to implement the State pattern the way I did, and you might like it too.

I've been ravenously consuming docs from the folks behind Rust, Bevy, and Rapier. I'm also reading blogs by folks who are making games in Bevy, and just consuming as much as I can because I'm excited about this.

I think Bevy's ECS system is quite elegant, and seems to be quite in-line with Rust's more functional nature. It's a nice breath of fresh air having worked on the mess that is Unreal Engine 5 with my lovely Capstone team for the last 2-3ish months. UE5 is definitely easier, and there's usually a checkbox or dropdown for what you want, but man. Bevy's cool, Bevy's simple. I'm in full control of what I add to my own objects, and it's all very piecemeal. I only take what I need. I'm objectively less productive on Bevy, but it *feels* better.

It feels a bit strange to always `Query` for entities by their components in a system, rather than bind code directly to an entity as a component the way MonoBehaviors work, but I don't hate it.

I just need to:

- Get more comfortable with the borrow checker,
- Learn enough about Bevy and Rust to keep `main()` nice and tidy,
- And learn about what `dyn` is and why it didn't like what I was doing. Rust is apparently a generic language!

I've been skimming the Rust Book and some Bevy examples and I've already got some ideas. I'll be taking another crack at this project, certainly.

When I learn how to write code in the spirit and constraints of the Rust language I suspect I'll like Bevy more.

leif larson